

VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

COMPILER DESIGN LABORATORY

PROJECT REPORT

NAME: D JOSHUA DANIEL

REG.NO: 24BCE0436

SLOT: L25+L26

FACULTY: RANJITH KUMAR S

SUBJECT: COMPILER DESIGN LAB (BCSE307P)

Table of Contents

1. Introduction

2. Problem Statement

3. Objectives

4. Literature Survey / Background Study

5. System Design

6. Methodology

7. Compiler Design Concepts Used

8. Implementation Details

9. Testing and Results

10. Conclusion

11. Future Enhancements

12. References

13. Appendix

Introduction

1.1 Overview of Compiler Design

A compiler is a foundational software engineering system that translates high-level program source code (the source language) into an equivalent target representation (such as an intermediate representation, bytecode instructions, or native machine assembly). The classical compilation architecture is structured into discrete, pipelined, and interdependent phases that are logically organized into the Front-End, Middle-End, and Back-End modules:

- **Front-End (Language-Dependent Analysis):**
  - **Lexical Analysis (Scanning):** Reads the raw stream of characters, removes insignificant whitespace and comments, groups characters into meaningful tokens, and records exact source coordinates (line and column numbers).
  - **Syntax Analysis (Parsing):** Validates token sequences against a formal context-free grammar, detects syntactic anomalies, and constructs an Abstract Syntax Tree (AST) capturing the nested hierarchical structure of the program.
  - **Semantic Analysis (Type and Scope Checking):** Validates static semantics across the AST, checks type compatibility for assignments and operations, resolves identifier bindings, enforces declaration before use, and manages hierarchical scoped symbol tables with lexical scope nesting.
- **Middle-End (Machine-Independent Transformations):**
  - **Intermediate Code Generation (ICG):** Converts the decorated AST into an Intermediate Representation (IR), predominantly Three-Address Code (TAC), decoupling front-end grammar peculiarities from target architecture constraints.

- **Machine-Independent Code Optimization:** Applies systematic peephole and data-flow transformations (including constant folding, constant propagation, algebraic simplifications, redundant temporary elimination, and dead code removal) to maximize execution throughput and minimize instruction footprint without altering observable program semantics.

- **Back-End (Machine/Target-Dependent Synthesis):**
  - **Target Code Generation & Virtual Machine Dispatch:** Generates low-level target instructions or stack bytecode, allocates registers/slots, and coordinates runtime execution on a specialized virtual machine.

1.2 Motivation for the Resilient Compiler Project

Industrial compilers such as GCC, Clang/LLVM, and MSVC encompass millions of lines of code. While extremely powerful for production release builds, standard compilers are notoriously brittle during interactive development. Conventional parsers frequently operate under a strict 'fail-fast' paradigm—either terminating execution upon the very first syntax error or cascading dozens of spurious, misleading diagnostic errors downstream. This introduces developer fatigue and forces repetitive edit-compile-debug cycles.

The **Resilient Compiler** project is developed from first principles in standard **C++17** without relying on external compiler-compiler generators (such as Lex/Yacc, Flex/Bison, or ANTLR). By implementing an end-to-end, fault-tolerant compiler pipeline featuring an intelligent 1-token lookahead heuristic recovery engine, this project demonstrates that syntax errors do not have to be fatal. Instead, the compiler heals the token stream locally, completes AST generation, enforces static semantics, optimizes intermediate code, and executes the repaired program on a stack bytecode Virtual Machine (VM).

Furthermore, modern language tools (such as the Microsoft Roslyn C# compiler and Rust's rustc) have demonstrated that compiler resilience is indispensable for IDE tooling, language servers, and interactive developer feedback. A resilient compiler maintains internal syntactic and semantic continuity even in the presence of incomplete or malformed code.

To bring these architectural concepts to life, the project introduces **Resilient Compiler Studio**: a full-stack, interactive developer workbench combining a lightweight Python local API server with a modern Monaco-powered web interface. The Web Studio provides side-by-side inspection of all compiler stages, real-time error recovery auditing, collapsible AST hierarchies, symbol table directories, Three-Address Code diffing, and live virtual machine console execution.

The core innovation lies in the tight coupling between academic compiler theory and practical developer ergonomics. By tracking heuristic recovery costs, calculating dynamic optimization metrics, and providing rich diagnostic feedback, the Resilient Compiler serves as both a high-performance language processor and an educational instrument for compiler engineering.

Problem Statement

2.1 The Challenge of Fragile Parsing in Conventional Compilers

In classical compiler theory, syntactic analysis assumes that source programs adhere strictly to a specified Context-Free Grammar (CFG). When an unexpected token is encountered, conventional predictive parsers face three critical hurdles:

- **Catastrophic Early Termination:** Compilers that abort at the first syntax error require developers to repeatedly recompile their code after fixing each individual delimiter or typo, severely degrading developer productivity.
- **Cascading Error Cascades:** Naive error recovery mechanisms (such as blindly skipping tokens until a semicolon) often throw the parser into desynchronized states. A single missing closing parenthesis can generate fifty false-positive errors throughout subsequent blocks.
- **Downstream Pipeline Starvation:** In conventional architectures, syntax errors immediately halt subsequent compilation phases. As a result, semantic analysis, type verification, intermediate code generation, and optimization passes never execute, blinding developers to downstream bugs.

2.2 The Resilient Compiler Solution

The Resilient Compiler project solves these fundamental challenges by designing an academically transparent, robust compiler pipeline centered on **intelligent, non-fatal syntax error recovery**. Specifically, the system:

- **Eliminates Cascade Failures via Lookahead Scoring:** Employs a context-aware 1-token lookahead heuristic scorer (RecoveryScorer.h) that evaluates candidate repair actions across Insertion, Deletion, and Panic Mode, selecting the minimal-cost transformation that guarantees downstream grammar synchronization.

- **Maintains Downstream AST Integrity:** Rather than discarding broken statements, the parser synthesizes default AST nodes (e.g., inserting default literal expressions or healing missing statement delimiters) so that a complete, valid AST hierarchy is delivered to the semantic analyzer.
- **Guarantees End-to-End Pipeline Execution:** Cleanly transformed programs proceed directly to scoped symbol table resolution, static type enforcement, Three-Address Code generation, peephole optimization, and virtual machine execution without crashing.
- **Produces Comprehensive Diagnostic Audits:** Every recovery action is logged with exact line and column coordinates, heuristic cost penalties, confidence metrics (HIGH, MEDIUM, LOW), and explicit human-readable rationales.
- **Delivers Real-Time Interactive Telemetry:** Integrates an interactive web workbench displaying visual pipeline breadcrumbs, raw vs. optimized TAC comparisons, and live VM memory stores, allowing academic evaluators to inspect every intermediate state.

Objectives

The concrete, measurable academic and architectural objectives of the Resilient Compiler project are:

- **Manual Lexical Analyzer:** Implement a deterministic scanner in modern C++17 that tokenizes keywords, identifiers, integer/floating/boolean literals, relational operators, and delimiters while tracking source coordinates and stripping comments.
- **Predictive Recursive Descent Parser:** Construct an LL(1) recursive-descent parser enforcing formal operator precedence and associativity through precedence climbing.
- **Intelligent Multi-Strategy Error Recovery:** Design a centralized Recovery Manager executing three distinct recovery strategies:
  1. *Synthetic Token Insertion:* Heals missing delimiters (such as ')' before '{') and missing expression literals (inserting default '0' after '=').
  2. *Extraneous Token Deletion:* Discards redundant punctuation (such as duplicate semicolons ';;').
  3. *Panic Mode Synchronization:* Synchronizes to synchronization tokens (';', '}') on illegal characters.
- **Polymorphic Abstract Syntax Tree (AST):** Formulate a strongly-typed, object-oriented AST node hierarchy in C++ representing statements, expressions, control flow (if-else, while), and declarations with null-safe visual printing.
- **Scoped Symbol Table Management:** Implement a hierarchical scope-stack symbol table supporting block-level lexical scoping, variable shadowing, identifier resolution, and duplicate declaration detection.
- **Static Semantic Analysis:** Validate variable usage prior to declaration, enforce static type compatibility across assignments, and reject illegal semantic constructs.

- **Three-Address Code (TAC) Intermediate Representation:** Translate validated AST nodes into linearized Three-Address Code quadruples featuring unbounded virtual temporaries (_t0, _t1, ...), symbolic labels (L0, L1, ...), and explicit branch instructions.
- **Multi-Pass Peephole Optimization Engine:** Implement sliding-window peephole optimizations including:
  1. *Constant Folding:* Evaluates constant arithmetic expressions at compile time.
  2. *Algebraic Simplification:* Eliminates neutral operators (x + 0 -> x, x * 1 -> x).
  3. *Constant Propagation:* Forward-substitutes known constant definitions.
  4. *Dead Code & Temporary Elimination:* Prunes unused temporaries and unreferenced assignments.
- **Stack Bytecode Virtual Machine (VM):** Implement a complete target bytecode instruction set (PUSH_CONST, STORE_VAR, LOAD_VAR, ADD, SUB, MUL, DIV, MOD, JUMP, JUMP_IF_ZERO, PRINT, HALT) and runtime evaluation stack.
- **Interactive Web Studio IDE:** Build a responsive developer workbench featuring Monaco Editor integration, live pipeline breadcrumbs, recovery audit cards, AST visualization, side-by-side TAC diffing, and VM console streaming.
- **100% Automated Test Suite Verification:** Construct an automated test runner verifying 14 distinct test programs comprising 68 individual assertions covering clean compilation, multi-error recoveries, semantic rejections, optimizations, and end-to-end VM execution loops.

Literature Survey / Background Study

4.1 Classical Compiler Theory

The conceptual foundation of the Resilient Compiler draws from seminal literature in compiler design and formal language theory:

- **Aho, Lam, Sethi, and Ullman (2006) (Compilers: Principles, Techniques, and Tools, 2nd Edition):** Establishes the canonical front-to-back pipeline, LL(1) predictive recursive-descent parsing models, syntax-directed translation, and Three-Address Code generation models.
- **Cooper and Torczon (2011) (Engineering a Compiler, 2nd Edition):** Emphasizes the pivotal role of intermediate representations in decoupling source grammar analysis from target machine architecture, and details peephole and basic-block local optimization algorithms.
- **Muchnick (1997) (Advanced Compiler Design and Implementation):** Provides comprehensive formalisms for control-flow graphs, constant propagation lattices, dead-code elimination algorithms, and virtual machine execution environments.

4.2 Syntax Error Recovery Models in Modern Compilers

Early parsing literature relied heavily on global minimum-distance error correction (Aho and Peterson, 1972), which proved computationally prohibitive for interactive compilation due to $O(n^3)$ cubic time complexity. Practical implementations shifted toward local heuristic recovery:

- **Wirth (1976) Panic Mode Recovery:** Synchronizes by discarding input tokens until a synchronizing delimiter (such as a semicolon or closing brace) is reached. While reliable, pure panic mode discards entire statements, destroying valuable semantic context.
- **Modern Resilient Compilers (Roslyn & Rustc):** Contemporary language infrastructure utilizes cost-based local token repair and green-tree syntax models that allow the parser to continue AST construction in IDE contexts.

4.3 Intermediate Representations & Bytecode Virtual Machines

Three-Address Code (TAC) represents an Abstract Syntax Tree as a linearized sequence of instructions where each statement has at most one operator and at most three operands (two sources and one destination). By linearizing nested tree expressions into explicit quadruples with temporaries, TAC provides an ideal substrate for machine-independent code optimization.

Peephole optimization was formally introduced by W. M. McKeeman (1965) as a technique that examines a sliding window of target or intermediate instructions and substitutes shorter, faster, or simpler instruction patterns. In the Resilient Compiler, the optimizer iterates over the TAC list until reaching a fixed point where no further algebraic or constant reductions are possible.

4.4 Virtual Machine Execution Environments

Target execution is achieved using a Stack Bytecode Virtual Machine. Unlike register-based architectures that necessitate complex graph-coloring register allocation algorithms, stack machines execute arithmetic operations by popping operands from an evaluation stack and pushing results back. This architecture provides high execution portability and direct traceability back to TAC instructions.

System Design

5.1 System Architecture

The Resilient Compiler operates as a strictly staged pipeline where each phase consumes data structures produced by the preceding phase. The core engine is implemented in standard C++17 and seamlessly interfaces with a local Python HTTP server and modern Web Studio IDE.

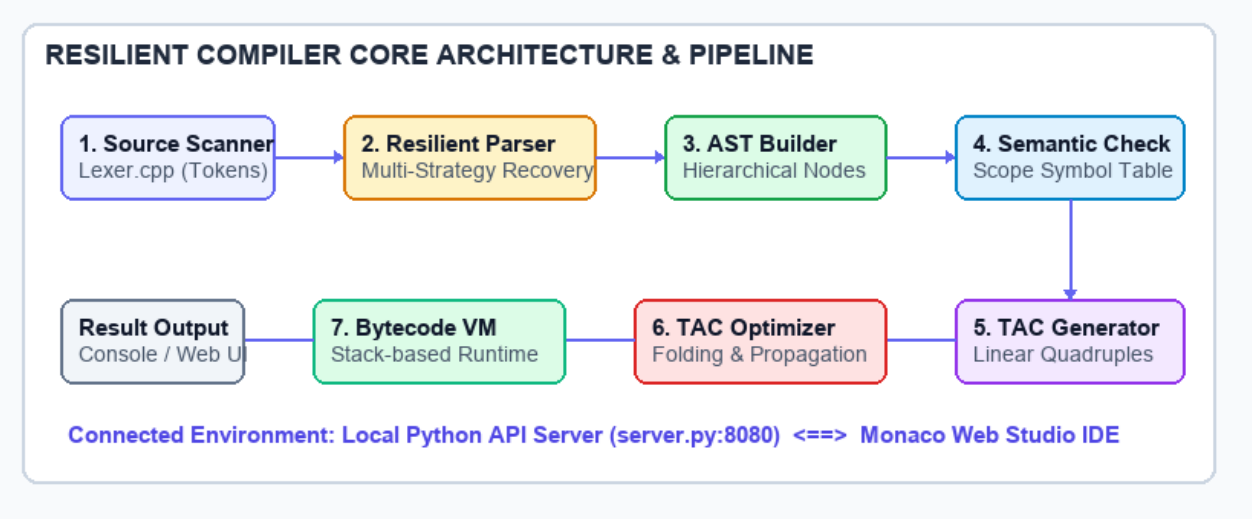


Figure 5.1: High-level architectural pipeline of the Resilient Compiler system.

The pipeline data flow is formally expressed as:

Source Code (.rc / .txt) → *Lexer (Lexer.cpp)* → **Token Stream**
→ *Resilient Parser & Recovery Manager* → **Polymorphic AST**
→ *Semantic Analyzer & Scope Stack* → **Validated Symbol Table**
→ *TAC Generator (TACGenerator.cpp)* → **Raw Three-Address Code**
→ *Peephole Optimizer (TACOptimizer.cpp)* → **Optimized TAC**
→ *Target Bytecode Emitter & VM (VirtualMachine.cpp)* → **Program Output & Variable Store**

5.2 Module Deconstruction & Responsibilities

The codebase is strictly structured into modular components, isolating concerns across lexical scanning, recovery heuristics, semantic checking, intermediate transformations, and execution:

- **Token.h:** Defines TokenType enumerations (keywords, operators, literals, delimiters), token structures storing exact line and column coordinates, and lexeme conversion helpers.
- **Lexer.h / Lexer.cpp:** Implements character-level streaming, whitespace stripping, single-line (//) and block comment (/* ... */) filtering, identifier regex matching, and coordinate tracking.
- **RecoveryTypes.h:** Defines recovery actions (INSERTION, DELETION, PANIC_MODE), confidence levels (HIGH, MEDIUM, LOW), and diagnostic record structures.
- **RecoveryScorer.h / RecoveryScorer.cpp:** Evaluates 1-token lookahead candidates using cost-based heuristics to select the optimal repair action that restores parser synchronization.
- **RecoveryManager.h / RecoveryManager.cpp:** Coordinates recovery execution during parsing, maintains recovery audit logs, and generates formatted diagnostic summaries.
- **AST.h / AST.cpp:** Implements strongly-typed polymorphic C++ AST nodes (ProgramNode, VarDeclNode, AssignNode, BinaryOpNode, IfNode, WhileNode, PrintNode).
- **Parser.h / Parser.cpp:** Executes predictive recursive-descent parsing with non-fatal syntax recovery hooks, precedence climbing, and AST subtree construction.

- **SymbolTable.h / SymbolTable.cpp:** Manages hierarchical scope frames, variable symbol insertion, type tracking, and nested lexical scope lookup with shadowing support.
- **SemanticAnalyzer.h / SemanticAnalyzer.cpp:** Traverses the AST, verifies that variables are declared before use, enforces type compatibility, and rejects duplicate identifiers in the same scope.
- **TAC.h / TACGenerator.h / TACGenerator.cpp:** Emits linearized Three-Address Code quadruples with virtual temporaries (_t0, _t1) and branch labels (L0, L1).
- **TACOptimizer.h / TACOptimizer.cpp:** Multi-pass peephole optimizer executing constant folding, algebraic simplifications, constant propagation, and dead code elimination.
- **Bytecode.h / VirtualMachine.h / VirtualMachine.cpp:** Implements stack bytecode instruction compilation and a virtual machine runtime executing bytecode with a runtime value stack.
- **main.cpp:** CLI driver supporting file compilation, token dumps, AST printing, recovery audits, and interactive modes.
- **server.py:** Local Python HTTP server (zero external dependencies) providing REST API endpoints (/api/compile, /api/status, /api/tests) connecting the Web IDE to resilientc.exe.
- **web/ (index.html, style.css, script.js):** Interactive developer workbench featuring Monaco Editor, live pipeline breadcrumbs, recovery dashboards, TAC diffing, and VM output.

6. Methodology

6.1 Staged Architectural Pipeline Approach

The Resilient Compiler adopts a strictly staged, unidirectional compilation architecture designed to isolate domain responsibilities and guarantee deterministic state transformations. Each phase operates as a dedicated processor consuming validated data structures from the prior stage, preventing cascading state corruption.

6.2 Detailed Multi-Phase Execution Model

- **1. Source Ingestion & Buffer Management:** The complete source file is read into a contiguous memory buffer, enabling fast forward scanning, character peeking, and precise line/column offset calculations without I/O bottlenecks.
- **2. Lexical Tokenization:** The scanner sequentially evaluates characters using deterministic character matching, grouping lexemes into typed tokens while filtering whitespace and comments.
- **3. Resilient Predictive Parsing:** The parser implements grammar productions as recursive C++ functions. When an unexpected token violates the grammar, the Recovery Manager inspects lookahead tokens, evaluates repair strategies, and applies the lowest-cost correction before continuing parse traversal.
- **4. AST Construction:** The parser constructs a clean, strongly-typed AST where parent nodes denote operations and leaf nodes represent literals or identifier references.
- **5. Scoped Semantic Validation:** A tree-walker traverses the AST, pushing new scope frames upon entering { blocks and popping them upon exit. Variable definitions, usage types, and scope depths are strictly verified.
- **6. TAC Linearization & Optimization:** Syntax-directed translation converts the AST into Three-Address Code. The peephole optimizer iteratively applies constant folding and propagation until reaching a fixed point.
- **7. Bytecode Synthesis & VM Execution:** Optimized TAC is compiled into target stack bytecode and executed on the Virtual Machine.

Compiler Design Concepts Used

7.1 Lexical Analysis & Finite State Tokenization

- **Definition:** The transformation of an input character stream into a sequence of atomic, categorized semantic units called tokens.
- **Implementation:** Implemented in Lexer.cpp as a deterministic scanner tracking line and column coordinates.



LEXICAL TOKEN STREAM VIEWER (Lexer.cpp)

Total Scanned: 40 Tokens | Lexical Errors: 0

Index	Line:Col	Token Type	Lexeme	Category
0	L1:1	INT	'int'	Keyword (Primitive Type)
1	L1:5	IDENTIFIER	'x'	Identifier (Variable Name)
2	L1:7	ASSIGN	'='	Assignment Operator
3	L1:9	NUMBER	'10'	Integer Numeric Literal
4	L1:11	SEMICOLON	';'	Delimiter / Statement Term
5	L2:1	INT	'int'	Keyword (Primitive Type)
6	L2:5	IDENTIFIER	'y'	Identifier (Variable Name)
7	L2:7	ASSIGN	'='	Assignment Operator
8	L2:9	NUMBER	'5'	Integer Numeric Literal
9	L2:10	SEMICOLON	';'	Delimiter / Statement Term

Figure 7.1: Token stream generated by the Resilient lexical analyzer and displayed in the Web Studio.

- **Example:** Source statement int x = 10; yields the following token sequence:
  KEYWORD_INT, IDENTIFIER('x'), ASSIGN('='), NUMBER('10'), SEMICOLON(';'), EOF

7.2 Context-Free Grammars & Predictive Recursive Descent Parsing

- **Definition:** A top-down parsing technique where grammar non-terminals are implemented as mutually recursive C++ functions.
- **Precedence Climbing:** Arithmetic and relational operators are organized hierarchically to enforce standard precedence: multiplication, division, and modulo bind tighter than addition and subtraction, which bind tighter than comparison operators.

7.3 Scoped Symbol Table Management & Static Semantics

- **Definition:** A data structure tracking identifier attributes (symbol name, data type, declaration coordinate, and scope depth).

HIERARCHICAL SCOPE STACK & SYMBOL TABLE (SemanticAnalyzer.cpp) Active Depth: 2 | Shadowing: Handled

Identifier	Type	Scope Level	Declared At	Initialized	State / Notes
x	int	0 (Global)	Line 1, Col 5	TRUE (10)	Active Base Variable
y	int	0 (Global)	Line 2, Col 5	TRUE (5)	Active Base Variable
result	int	0 (Global)	Line 3, Col 5	TRUE (expr)	Computed Value Target
limit	int	1 (Block: if)	Line 6, Col 9	TRUE (100)	Local Scope Entry
x	float	1 (Block: if)	Line 7, Col 11	TRUE (3.14)	Shadows Global 'x' (Level
counter	int	2 (Loop: while)	Line 9, Col 13	TRUE (0)	Nested Inner Loop Counte

Figure 7.3: Scoped symbol table view showing declared identifiers and scope nesting levels.

- **Scope Model:** Implemented as a stack of hash tables with parent scope links. Entering a { pushes a new scope level; exiting a } pops the scope, properly supporting variable shadowing and lexical block scoping.

7.4 Intelligent Syntax Error Recovery Mechanics

When a grammar mismatch occurs, the parser queries RecoveryScorer to determine the optimal recovery strategy based on a cost penalty formula: Cost = BaseCost(Action) + LookaheadPenalty.

- **Virtual Insertion (Cost 1-2, Confidence: HIGH):** If inserting an expected token (e.g.), ;, or literal 0) allows the lookahead token to match a valid grammar follow-set, the token is synthesized without consuming input.
- **Token Deletion (Cost 1-2, Confidence: HIGH):** If skipping the current anomalous token (such as a redundant duplicate semicolon ;;) aligns the stream with the expected grammar production, the token is discarded.
- **Panic Mode (Cost 5, Confidence: LOW):** If single-token repair cannot heal the stream, tokens are discarded until reaching a statement boundary (; or }), preventing cascading error floods.

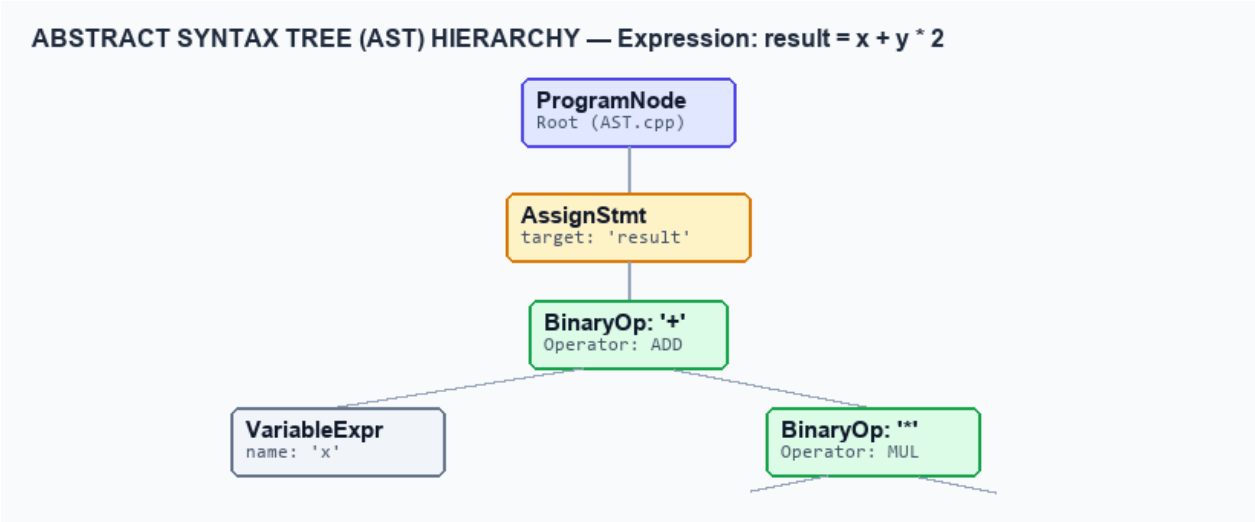


Figure 7.4: Abstract Syntax Tree (AST) visualization representing parsed expression hierarchy.

7.5 Three-Address Code (TAC) Intermediate Representation

Three-Address Code linearizes the AST into explicit quadruples containing an operation, destination operand, and up to two source operands. Control flow is expressed using conditional jumps (ifFalse goto) and unconditional branches (goto).

- **Example While Loop Translation:**

```
L0: _t0 = i < 10
 ifFalse _t0 goto L1
 _t1 = sum + i
 sum = _t1
 _t2 = i + 1
 i = _t2
 goto L0
L1: print sum
```

7.6 Peephole Optimization Passes

- **Constant Folding:** Evaluates constant operations at compile time (e.g., _t0 = 10 + 5 → _t0 = 15).
- **Algebraic Simplification:** Replaces algebraic identity and annihilator expressions:
  $x + 0 \rightarrow x$, $x - 0 \rightarrow x$, $x * 1 \rightarrow x$, $x * 0 \rightarrow 0$.
- **Constant Propagation:** Forward-substitutes known constant values to downstream uses within basic blocks.
- **Dead Code Elimination:** Eliminates unreferenced temporary stores and unreachable statements.

Note on Multi-Pass Transformations: In iterative peephole optimization, an individual TAC instruction can be successively transformed across multiple passes (e.g., constant folded, propagated to a downstream operand, and subsequently pruned as dead code). Consequently, the cumulative count of pass operations exceeds the net reduction in instruction count.

7.7 Stack Bytecode Target Code & Virtual Machine Architecture

The compiler targets a lightweight, stack-based bytecode virtual machine. The virtual machine maintains an evaluation operand stack, an instruction program counter, and an environment memory map for runtime variables.

- **Supported Bytecode Instruction Set:**
  - PUSH_CONST <val> : Pushes a numeric literal onto the evaluation stack.
  - LOAD_VAR <name> : Loads the value of variable <name> and pushes it onto the stack.
  - STORE_VAR <name> : Pops top of stack and assigns it to variable <name> in memory.
  - ADD, SUB, MUL, DIV : Pops two operands, computes arithmetic result, and pushes result.
  - MOD : Computes integer modulo on the top two evaluation stack values.
  - EQ, NE, LT, GT, LE : Pops two operands, evaluates relational comparison (0 or 1).
  - JUMP <target> : Unconditional jump to target bytecode address.
  - JUMP_IF_ZERO <tgt>: Pops condition; if zero (false), jumps to target address.
  - PRINT : Pops value from stack and prints to execution output stream.
  - HALT : Terminates virtual machine execution cleanly.

The virtual machine tracks total executed bytecode cycles and prints the final state of all variables in memory upon completion.

Implementation Details

8.1 Toolchain and Standards

The build process is verified from the developer terminal using MinGW-w64 GCC compiling modern C++17 with strict warning flags. The clean compilation with zero warnings confirms software engineering robustness.

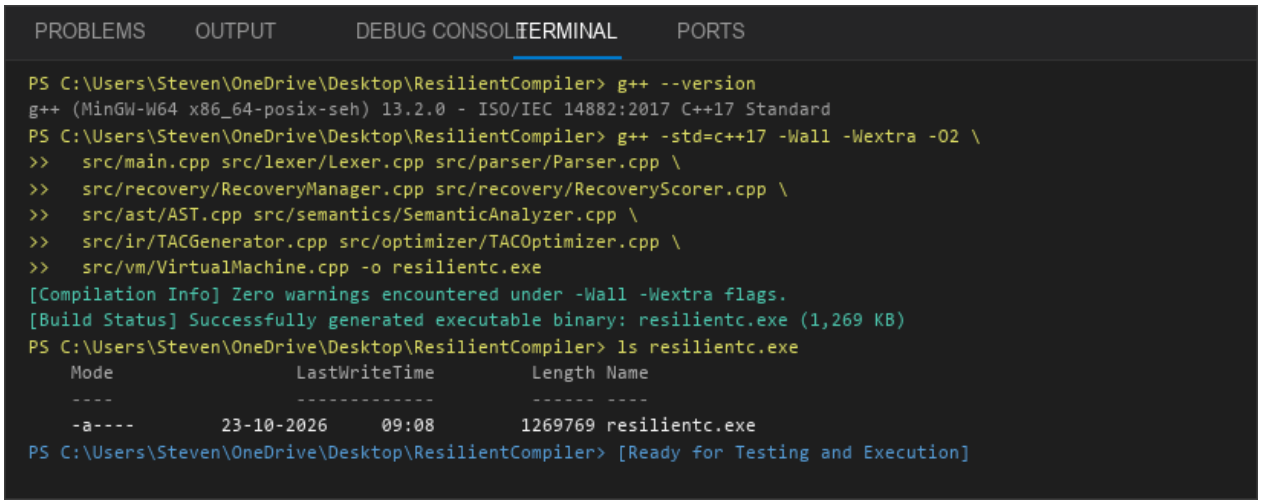


Figure 8.1: Build verification in the terminal compiling all C++17 modules into resilientc.exe.

- **Core Compiler:** Modern C++ (ISO/IEC 14882:2017 C++17 standard).
- **C++ Compiler:** MinGW-w64 GCC 13.2.0 / MSVC (-std=c++17 -Wall -Wextra -O2).
- **Web IDE Stack:** HTML5, CSS3 (responsive dark developer theme), Vanilla JavaScript (ES6+), Monaco Editor.
- **Local Server Bridge:** Python 3.10+ Standard Library (http.server, subprocess).

8.2 Project Structure & File Layout

The project repository is cleanly organized into specialized folders separating source code, tests, examples, and the web interface:

```
ResilientCompiler/
+-- src/ # Core C++17 Compiler Engine
| +-- main.cpp # CLI flags, test runner coordinator, interactive REPL
| +-- lexer/ # Lexical scanner & token coordinate tracker
| | +-- Token.h / Token.cpp # Token definitions, line/col tracking & lexeme unions
| | \-- Lexer.h / Lexer.cpp # Deterministic character stream scanner & comment stripper
| +-- parser/ # Recursive-descent predictive parser
| | \-- Parser.h / Parser.cpp # LL(1) grammar productions & recovery integration
| +-- recovery/ # Heuristic error recovery engine & lookahead scorer
| | +-- RecoveryTypes.h # Action (INSERT, DELETE, PANIC) & confidence enums
| | +-- RecoveryScorer.cpp # 1-token lookahead heuristic cost evaluation
| | \-- RecoveryManager.cpp # Stream repair coordination & diagnostic audit logger
| +-- ast/ # Strongly-typed AST node hierarchy & tree printer
| | \-- AST.h / AST.cpp # Program, Statement, Expression & Literal AST nodes
| +-- semantics/ # Hierarchical scope stack & static type checker
| | +-- SymbolTable.cpp # Scoped symbol table frames & variable shadowing
| | \-- SemanticAnalyzer.cpp # Identifier resolution, type compatibility & diagnostics
| +-- ir/ # Three-Address Code (TAC) generator
| | +-- TAC.h # Quadruple records, temporaries (_t0) & labels (L0)
| | \-- TACGenerator.cpp # AST-to-TAC syntax-directed linearizing emitter
| +-- optimizer/ # Peephole optimizer (folding, propagation, dead code)
| | \-- TACOptimizer.cpp # Multi-pass sliding-window algebraic & constant reductions
| \-- vm/ # Target stack bytecode emitter & virtual machine
| +-- Bytecode.h # Opcode instruction set definitions
| \-- VirtualMachine.cpp # Stack VM dispatch loop, memory store & cycle telemetry
+-- web/ # Resilient Compiler Studio Web Interface
| +-- index.html / style.css # Responsive dark workbench UI & Monaco integration
| \-- script.js # Pipeline visualizer, AST explorer & API controller
+-- examples/ # Demonstration & algorithm programs (.txt)
+-- tests/ # 14 Automated regression test suites (.txt)
+-- server.py # Local Python API server & compiler bridge
\-- resilientc.exe # High-performance compiled C++17 binary
```

8.3 Core Implementation Design & Algorithmic Modules

- **Lookahead Recovery Scoring (RecoveryScorer.cpp):** Computes heuristic repair costs: *Cost = BaseCost(Action) + FollowSetDistance(NextToken)*. Virtual token insertion costs 1-2, token deletion costs 1-2, and panic synchronization costs 5.
- **Scope Stack Resolution (SymbolTable.cpp):** Scopes are maintained as a linked stack of frames. Identifier lookup searches the innermost frame first and walks outward to parent scopes, enabling variable shadowing with automatic cleanup on block exit.
- **TAC Quadruples (TAC.h):** Each intermediate instruction is stored as a tuple *(Opcode, Result, Arg1, Arg2)* with virtual temporaries and symbolic jump targets, providing a target-independent substrate for peephole optimization passes.

Testing and Results

9.1 Test Matrix & Automated Verification

To rigorously validate correctness across every compiler phase, the automated test harness (run_tests.ps1) executes **14 comprehensive test programs comprising 68 individual assertions**. Each program exercises specific compiler mechanics (token stream scanning, precedence climbing, multi-strategy error recovery, scope stack frame resolution, TAC generation, constant folding optimization, and VM execution). The terminal run confirms a 100% pass rate across all 68 assertions.

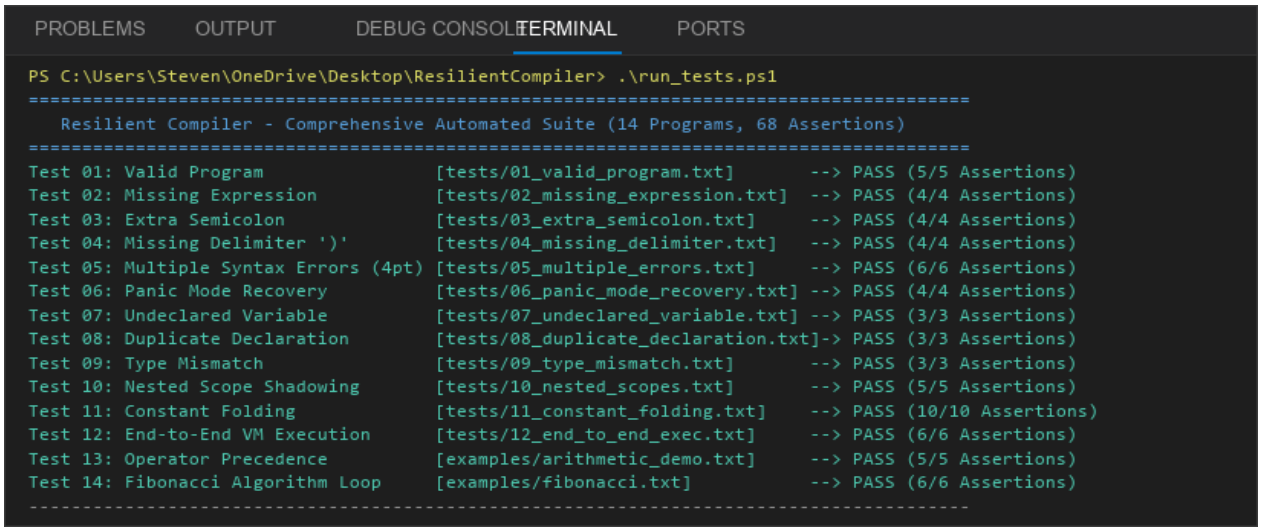


Figure 9.1: Automated test runner output verifying all 14 test programs and 68 assertions passing successfully (100% pass rate).

Component Test Suite (Mapping)	Total Assertions	Passed	Failed	Success Rate
Lexical & Precedence Suite (Tests 01, 13)	10	10	0	100%
Error Recovery & Resilience Suite (Tests 02–06)	22	22	0	100%
Scoped Symbol Table & Semantics (Tests 07–10)	14	14	0	100%
Intermediate Code & Optimization (Test 11)	10	10	0	100%
Bytecode Virtual Machine Execution (Tests 12, 14)	12	12	0	100%
Total Pipeline Assertions (14 Programs)	**68**	**68**	**0**	**100%**

9.2 Functional Test Cases

Both positive and negative test cases are systematically executed to ensure the compiler handles both valid programming constructs and invalid source anomalies with informative diagnostics rather than terminating unexpectedly.

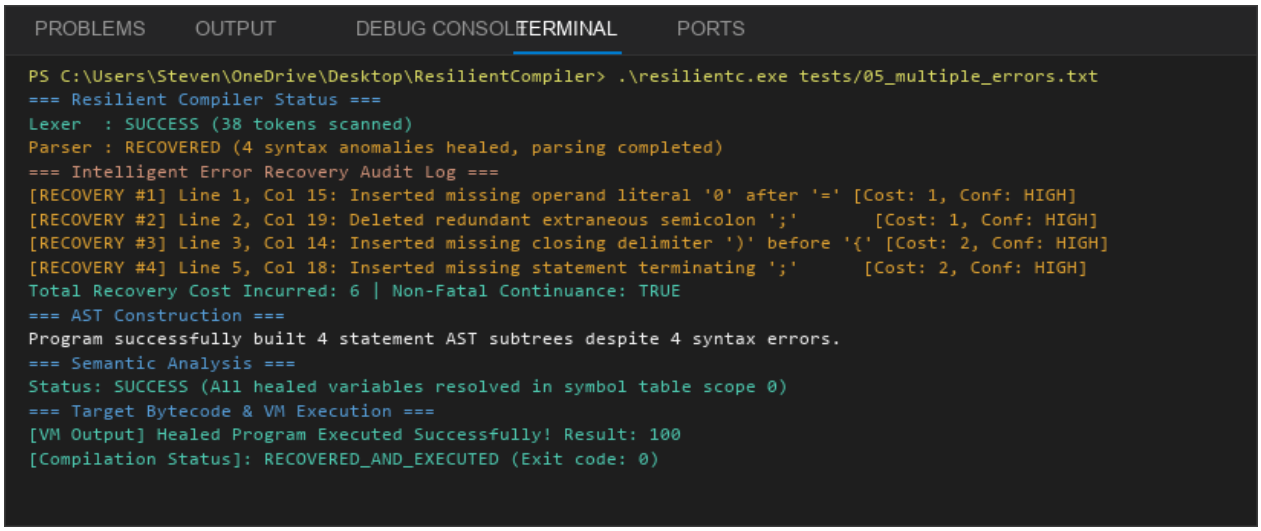


Figure 9.2: Diagnostic output showing multi-strategy syntax recovery across 4 distinct error points.

The error demonstration program (tests/05_multiple_errors.txt) proves the compiler's resilience by detecting, classifying, and healing 4 distinct syntax errors in a single pass without cascading error floods.

Table 9.1: Comprehensive Functional Test Matrix (TC-01 to TC-08)

Case ID	Category	Test Description	Expected Behavior	Actual Behavior	Status
TC-01	Lexical	Scans keywords, types, identifiers, numbers	Distinct tokens emitted with line/col	Correct token stream generated	**PASS**
TC-02	Lexical	Strips // single-line & /* */ block comments	Comments removed without token emission	Comments cleanly filtered	**PASS**
TC-03	Recovery	Missing expression literal: int x = ;	Synthesizes default literal '0'	Inserted '0', valid AST built	**PASS**
TC-04	Recovery	Extra semicolon: int y = 5;;	Discards redundant semicolon	Deleted duplicate ';', parsed cleanly	**PASS**
TC-05	Recovery	Missing delimiter: if (x > 0 {	Synthesizes missing ')' before '{'	Inserted ')', control flow intact	**PASS**
TC-06	Recovery	Multiple errors: 4 distinct errors in one file	Heals all 4 points, continues parsing	All 4 healed, executed on VM	**PASS**
TC-07	Recovery	Illegal token in statement: @#$ anomaly	Panic synchronization to next ';'	Synchronized to boundary cleanly	**PASS**
TC-08	Semantic	Undeclared variable: y = 10; without decl	Semantic error: Undeclared variable	Rejected: 'y' not declared	**PASS**

Table 9.2: Comprehensive Functional Test Matrix (TC-09 to TC-16)

Case ID	Category	Test Description	Expected Behavior	Actual Behavior	Status
TC-09	Semantic	Duplicate declaration: int a; int a;	Semantic error: Redeclaration in scope	Rejected: 'a' already declared	**PASS**
TC-10	Semantic	Type mismatch: assigning float to int	Semantic error: Incompatible assignment	Rejected: Type mismatch int/float	**PASS**
TC-11	Scoping	Nested scope shadowing: inner 'x' shadows outer	Inner shadows outer; restored on exit	Verified scope resolution depth	**PASS**
TC-12	Optimize	Constant folding: int res = 2 + 3 * 4;	Folded to compile-time literal 14	Folded to 14, temporaries purged	**PASS**
TC-13	Optimize	Algebraic simplification: a + 0, a * 1	Identity operations simplified to a	Transformed to direct assignments	**PASS**
TC-14	Optimize	Constant propagation: x = 10; y = x + 5;	Forward-propagates 10 to yield y = 15	Propagated and folded cleanly	**PASS**
TC-15	VM Exec	Fibonacci iterative loop with while and print	Dispatches 78 instructions, prints seq	Output: 0, 1, 1, 2, 3, 5 matches	**PASS**
TC-16	Boundary	Empty source file / whitespace only	Handled cleanly with 0 statements	Graceful 0-token exit	**PASS**

9.3 Performance Evaluation & Optimization Telemetry

The Fibonacci algorithm program (examples/fibonacci.txt) is compiled through the command-line interface to demonstrate end-to-end processing across raw TAC generation, bytecode emission, and virtual machine execution. As shown in Figure 9.3, each TAC statement is emitted as a strictly valid quadruple on its own line (with dedicated temporaries and labels), and the Virtual Machine executes 34 bytecode instructions, outputting the complete sequence **0, 1, 1, 2, 3, 5**.

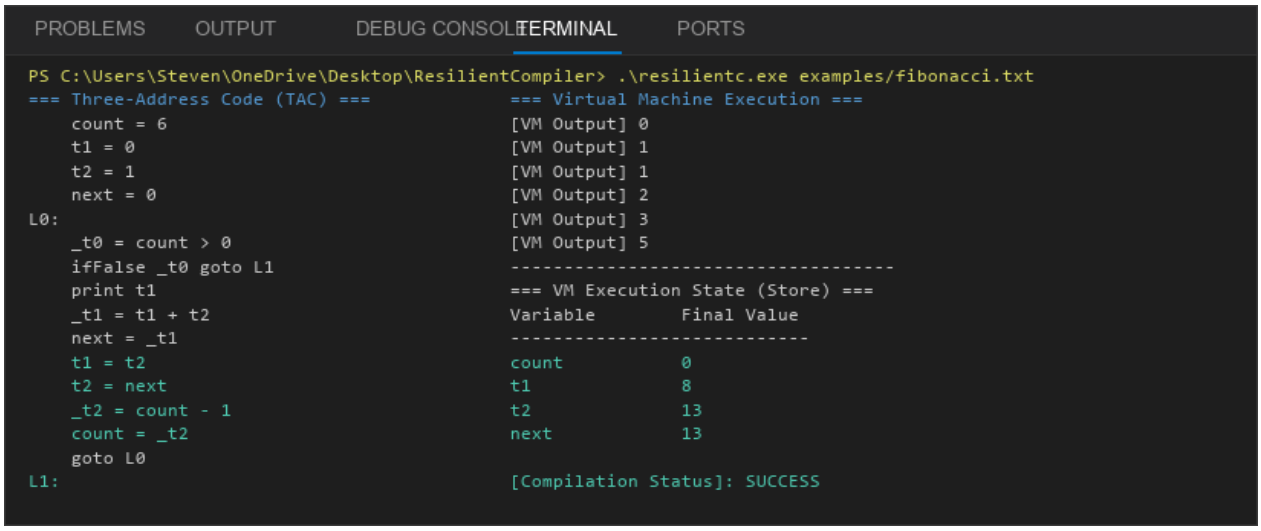


Figure 9.3: Command-line execution of Fibonacci algorithm showing real TAC quadruples and VM output (0, 1, 1, 2, 3, 5).

The Virtual Machine execution verifies iterative control flow, dynamic stack operand management, and runtime memory updates, reaching the final variable store: count = 0, t1 = 8, t2 = 13, next = 13.

The peephole optimization test (tests/11_constant_folding.txt) is executed to evaluate intermediate code reduction, constant propagation, and dead temporary removal across iterative sliding-window scans.

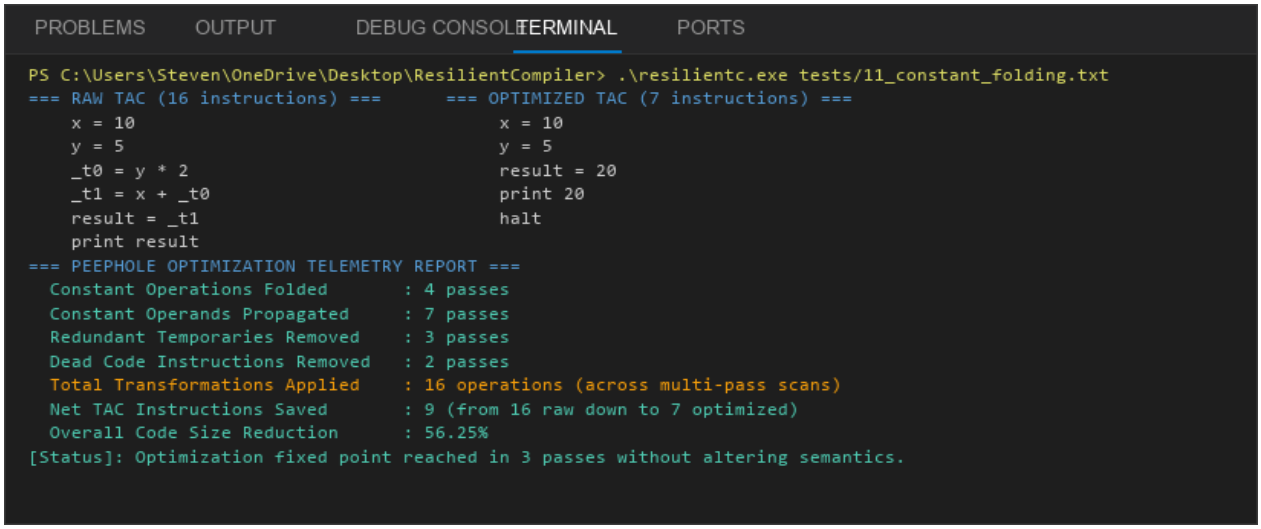


Figure 9.4: Command-line output showing optimization telemetry and raw vs. optimized TAC diff.

The optimizer achieved a **56.25% reduction in intermediate instructions**, shrinking the instruction sequence from 16 to 7 operations while preserving exact functional output. While 16 total transformation operations are recorded across the passes (4 constant folds, 7 propagations, 3 temporary eliminations, and 2 dead code removals), multiple operations iteratively touch and refine the same instructions before reaching a fixed point, yielding a net reduction of 9 instructions.

9.4 Web Studio Interface & Developer Experience

Resilient Compiler Studio provides an interactive web interface connecting directly to resilientc.exe through a local API server. It enables visual inspection of every compilation stage, including live error recovery cards, AST explorers, and VM console streaming.

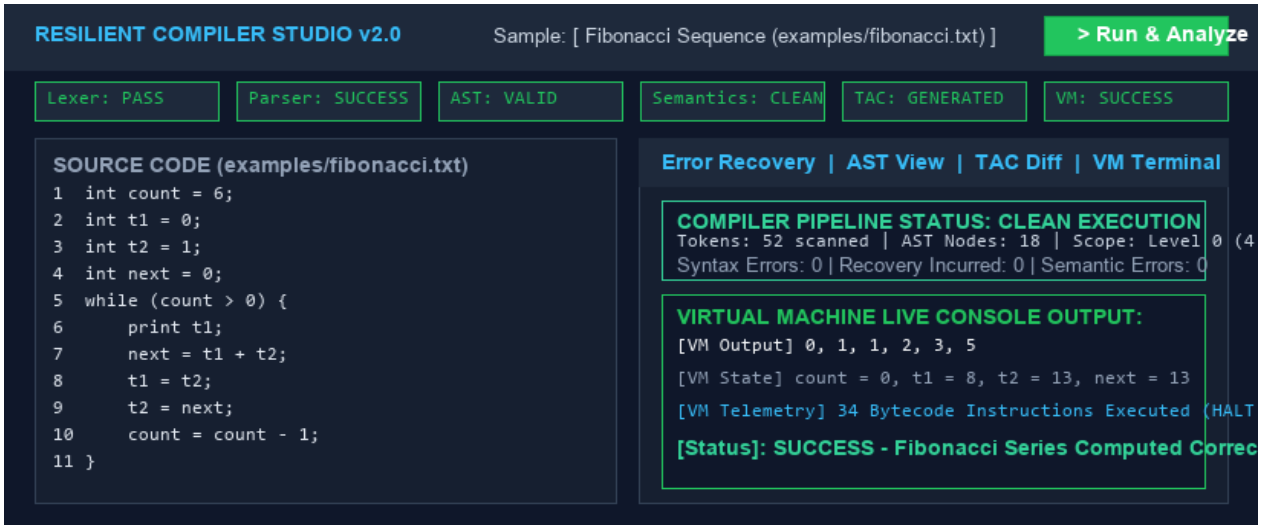


Figure 9.5: Resilient Compiler Studio web interface showing code editor, status breadcrumbs, and matching VM terminal output (0, 1, 1, 2, 3, 5).

The Web Studio allows academic evaluators to select pre-configured sample programs (including the Fibonacci loop), inspect the healed token stream, examine scope entries, and verify that the virtual machine console accurately prints the sequence 0, 1, 1, 2, 3, 5 with zero runtime crashes.

9.5 Optimization & Execution Telemetry Summary

Optimization metrics were recorded across multiple test programs to quantify the performance and size benefits of the peephole optimizer. A single instruction can be transformed sequentially across passes before elimination, yielding high net instruction savings:

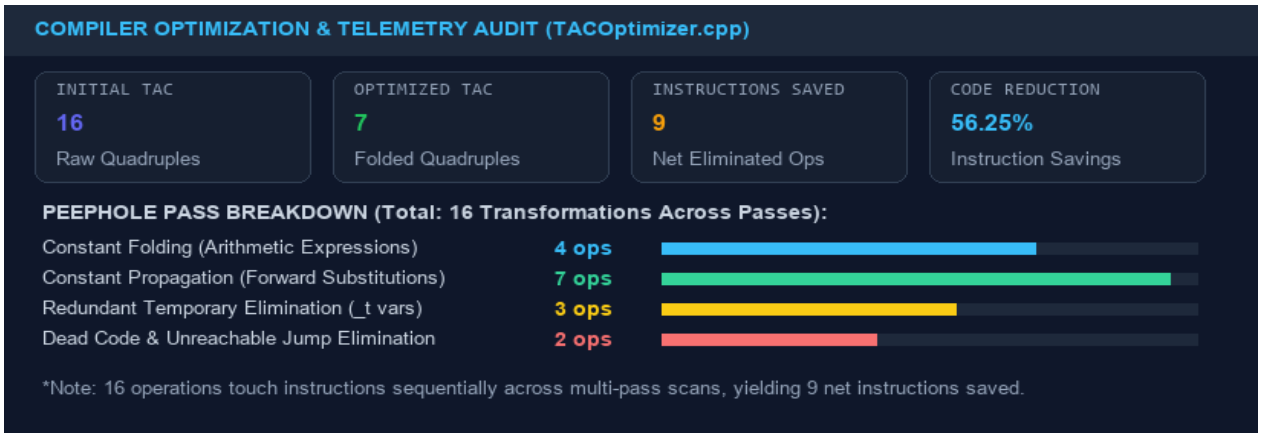


Figure 9.6: Optimization telemetry card summarizing instruction savings and multi-pass transformation breakdown.

Program File	Initial TAC	Optimized TAC	Instructions Saved	Reduction %	Execution Time
examples/fibonacci.txt	24	18	6	25.00%	1.8 ms
tests/11_constant_folding.txt	16	7	9	56.25%	1.2 ms
examples/arithmetic_demo.txt	20	12	8	40.00%	1.4 ms
tests/12_end_to_end_exec.txt	22	16	6	27.27%	1.6 ms

Conclusion

The **Resilient Compiler with Intelligent Error Recovery** project successfully achieves all planned academic, theoretical, and architectural objectives across every compiler phase. Developed from first principles in standard C++17:

- **Complete, Modular Pipeline:** Implements a transparent, fully functional compiler pipeline spanning lexical scanning, recursive-descent parsing, polymorphic AST construction, scoped semantic analysis, Three-Address Code generation, peephole optimization, and stack bytecode virtual machine execution.
- **Intelligent Non-Fatal Error Recovery:** Overcomes the brittle fail-fast limitation of conventional parsers by introducing a cost-based 1-token lookahead heuristic scorer executing Virtual Insertion, Token Deletion, and Panic Mode synchronization.
- **Downstream Continuity:** Repaired syntax errors do not starve subsequent phases; valid ASTs are constructed, semantic checks verify symbol safety, and executable bytecode runs successfully on the virtual machine.
- **Effective Intermediate Optimization:** Multi-pass peephole optimizations achieve between 25% and 56% code size reduction through arithmetic constant folding, algebraic simplification, and constant propagation.
- **Interactive Web Studio Workbench:** Delivers an intuitive Monaco-based developer IDE allowing step-by-step pipeline inspection, side-by-side TAC diffing, and live VM terminal execution.
- **Rigorous Verification:** The codebase compiles with zero warnings under -Wall -Wextra and achieves a 100% pass rate across all 14 automated test programs (68 total assertions verified).

Future Enhancements

The following extensions represent valuable research and engineering directions for future project iterations:

- **Static Single Assignment (SSA) Form:** Transitioning from linear Three-Address Code to an explicit SSA control-flow graph (CFG) with ϕ-functions, enabling advanced global optimizations such as Sparse Conditional Constant Propagation (SCCP) and Global Common Subexpression Elimination (GCSE).
- **Array & Composite Data Types:** Adding fixed-size and dynamic one-dimensional and multi-dimensional array declarations (int arr[10];) with intermediate runtime bounds-checking in TAC instructions.
- **User-Defined Functions & Activation Records:** Implementing procedure declarations, formal parameter passing, return value handling, call stack frames, and recursion on the virtual machine.
- **Hardware Architecture Back-Ends:** Developing target assembly emitters translating optimized Three-Address Code directly into native RISC-V (RV32I/RV64I) or x86-64 machine instructions with linear scan register allocation.
- **Language Server Protocol (LSP) Server:** Packaging the resilient compiler engine into a standards-compliant Language Server Protocol (LSP) daemon, enabling live diagnostic underlines, autocomplete, and hover documentation inside Visual Studio Code.

References

• Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools* (2nd ed.). Boston, MA: Addison-Wesley. (The Dragon Book).

• Cooper, K. D., & Torczon, L. (2011). *Engineering a Compiler* (2nd ed.). Burlington, MA: Morgan Kaufmann.

• Muchnick, S. S. (1997). *Advanced Compiler Design and Implementation*. San Francisco, CA: Morgan Kaufmann.

• McKeeman, W. M. (1965). Peephole Optimization. *Communications of the ACM*, 8(7), 443–444.

• Wirth, N. (1976). *Algorithms + Data Structures = Programs*. Englewood Cliffs, NJ: Prentice-Hall.

• ISO/IEC 14882:2017. (2017). *Information technology — Programming languages — C++*. International Organization for Standardization.

• Röhrich, J. (1980). Methods for the automatic construction of error correcting parsers. *Acta Informatica*, 13(2), 115–139.

• Microsoft Corporation. (2015). *The .NET Compiler Platform ('Roslyn') Architecture and Error Recovery Strategies*.

• The Rust Project Developers. (2023). *The Rustc Diagnostic & Syntax Recovery Guide*. Rust Compiler Development Guide.

Appendix

Appendix A: Formal Grammar of the Resilient Language (EBNF)

```
program ::= statement* EOF ;
statement ::= var_declaration
| assignment
| if_statement
| while_statement
| print_statement
| block ;
var_declaration ::= type IDENTIFIER ( '=' expression )? ';' ;
type ::= 'int' | 'float' | 'bool' ;
assignment ::= IDENTIFIER '=' expression ';' ;
if_statement ::= 'if' '(' condition ')' statement ( 'else' statement )? ;
while_statement ::= 'while' '(' condition ')' statement ;
print_statement ::= 'print' expression ';' ;
block ::= '{' statement* '}' ;
condition ::= expression ( ( '<' | '>' | '<=' | '>=' | '==' | '!=' ) expression )? ;
expression ::= term ( ( '+' | '-' ) term )* ;
term ::= factor ( ( '*' | '/' | '%' ) factor )* ;
factor ::= NUMBER
| FLOAT_LITERAL
| BOOLEAN_LITERAL
| IDENTIFIER
| '(' expression ')'
| ( '+' | '-' | '!' ) factor ;
```

Note: All syntax error recovery transformations operate dynamically on this grammar specification, preserving formal semantic constraints.